

\$ cat notes.md

DevOps & Cloud [SWE] — Class 13 Notes

LECTURE TITLE The Five Kubernetes Service Types (*see the note below — the folder name doesn't match what was taught*)

DATE/TIME 15 September 2026, 11:00 AM

DURATION 120 minutes

INSTRUCTOR Nensi Ravaliya

BIG IDEA A pod's IP changes every time it restarts. A Service is the stable name in front of it — and *which* Service type you pick is entirely about who needs to reach it.

WHAT'S INSIDE — 14 SECTIONS

- 00 ⚡ The 60-Second Version
- 01 🔄 Recap — The Whole of Kubernetes So Far
- 02 🗉 Labels & Selectors — Where Each One Goes
- 03 📡 The Four Ports — Untangled
- 04 🏠 1 ClusterIP — The Default (Inside Only)
- 05 🚪 2 NodePort — Open A Door On Every Node
- 06 ⚖️ 3 LoadBalancer — Production's Answer
- 07 🌐 4 ExternalName — A Signpost, Not A Service
- 08 🚫 5 Headless — And Why StatefulSets Need It
- 09 🏠 FQDN — The Full Address Of A Service
- 10 🎯 Which Service Type — The Decision Table
- 11 📄 Command Cheat-Sheet
- 12 📝 Homework
- 13 📄 Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. theme: Signal Blue

⚠ WATCH OUT

Read this first.

The folder name is wrong. It says *"Storage, HPA & Probes"*, and so does the transcript header. **None of those were taught.** The whole class was **Kubernetes Services** — all five types, with a live demo of each. Storage, HPA and Probes are still ahead of you.

The last ~25 minutes are largely unusable — a failed DNS experiment plus lab time, transcribed as noise, Portuguese, Spanish and Indonesian fragments. What survived is reconstructed below; what didn't is flagged rather than guessed.



00 ⚡ The 60-Second Version

1. **Every time a pod restarts, it gets a new IP.** That single fact is why Services exist.
2. **A Service is a stable name** that finds its pods by **label**, not by IP — so pods can be destroyed and replaced freely.
3. **Five types, one question: who needs to reach this?** **ClusterIP** (inside only) · **NodePort** (outside, via a port) · **LoadBalancer** (outside, via a URL) · **ExternalName** (points at an external DNS name) · **Headless** (no IP at all — for StatefulSets).
4. **clusterIP: None = headless.** That's literally how you declare it, and it's an interview question.
5. **StatefulSet gives pods stable names** — `mysql-0`, `mysql-1`. Delete `mysql-0` and `mysql-0` comes back. A Deployment would give you a new random name.
6. **port is the Service's port.** **targetPort** is the pod's port. **nodePort** is the node's port. Three different numbers.
7. **curl from your laptop terminal can't reach a ClusterIP** — your terminal is on the host network, not the cluster network.
8. **FQDN pattern:** `service.namespace.svc.cluster.local`.



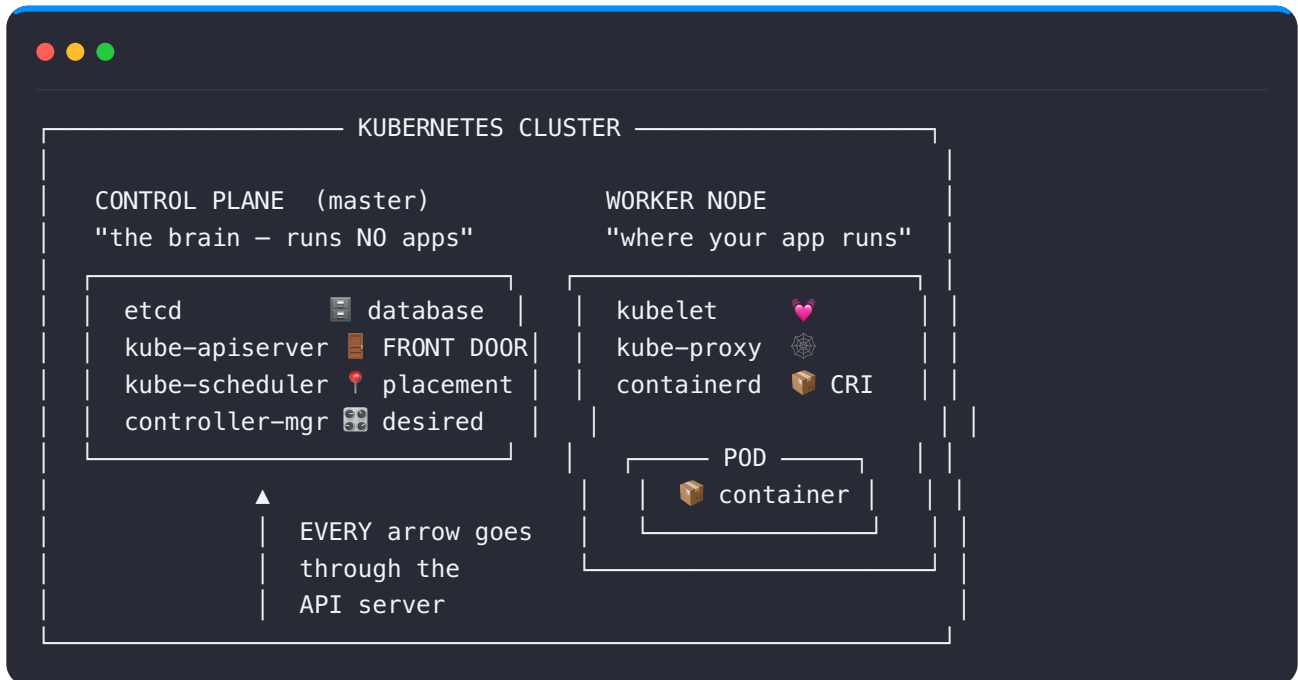
01 🔄 Recap — The Whole of Kubernetes So Far

She spent 20 minutes drilling the class, because *"at the end of the course, Kubernetes is ended and then you need to learn everything again. I don't want that confusion."*

NOTE

This is her third pass over the architecture (Class 9 introduced it, Class 10 walked the [kubectl apply](#) journey, this one drills it as Q&A). These notes re-teach it in full each time rather than pointing back — repetition is the point, and hearing it a third way is often what makes it stick.

The whole picture, one more time



The drill, question by question

Question	Answer
Master node components?	4 — etcd, scheduler, controller, API server
Who is the "friend" of your cluster?	API server
Can components talk directly?	✗ No — "Through which component? API server."
Worker node components?	2 — kubelet, kube-proxy (+ CRI)
Default container runtime?	containerd
kubelet's job?	"Communication with the control plane" + "manage the pod — create a pod, delete a pod"
etcd?	"key-value pair database"
API server's job?	communication hub + front door + validates your YAML
Scheduler's job?	schedule new pods
Controller's job?	manage replicas and deployments

Each component, properly — not just a table row

etcd — the only place anything is stored

SAID IN CLASS

"Key-value pair database." Everything about your cluster lives here: cluster state, pods, services, replica sets, deployments, **persistent volumes and persistent volume claims** — she listed those explicitly this time.

✓ "Anything inside a cluster you want to save — there is only one storage, which we call etcd."

kube-apiserver — three jobs, not one

She pushed the class past the one-word answer and got all three:

1. COMMUNICATION HUB → every component talks through it, never to each other
2. FRONT DOOR / GATEKEEPER → nothing enters the cluster without it
3. VALIDATION → it checks your YAML and creates the API object

 "Whatever we write in our YAML file — who is validating it? Who tells you there's a typo? That's what the API server is doing."

kube-scheduler — one sentence

SAID IN CLASS

"Scheduler's main job — it's very easy: new pods." It decides **which node** each new pod lands on, based on the resources that node has free.

controller-manager — why it exists at all

INTERVIEW ALERT

She asked "why do we need a controller? We don't need a controller." and made them argue it:

✓ "Because we need to manage the replicas, because we need to manage the deployments. In a deployment we have three replicas... we need to manage those replicas. That's why we have a controller."

And there isn't one controller — there are many: "ReplicaSet controller, deployment controller, node controller — lots of controllers are there."

kubelet — two jobs

SAID IN CLASS

"Communication with the control plane" + "manage the pod — create a pod, delete a pod."

kube-proxy + CRI

kube-proxy handles networking on the node. **CRI = Container Runtime Interface**, and *"which container runtime are we using by default? containerd."*

The bit worth keeping — how the components connect to things you've actually typed:

```
kubectl get nodes      → where does that data come from? → etcd
kubectl get deployment → where does that data come from? → etcd
a typo in your YAML    → who catches it?                → API server
```

 *"Whatever we write in our YAML file — who is validating it? Who tells you there's a typo? That's what the API server is doing. API server validates your file and creates an API object."*

LOCK THIS IN

Why she keeps re-drilling this, in her own words: *"You've got an idea now — we created ReplicaSets, DaemonSets, Deployments, Services. Have you got an idea how they are connected with each one? Maybe the controller — you get to know, okay, we're creating replicas, we're creating deployments, that's why the controller is there. etcd you can't see directly, but whatever you're storing is inside etcd."*

 **That's the actual goal of the recap:** not to memorise six names, but to connect each one to a command you've already typed. `kubectl get nodes` → etcd answered. A YAML typo → the API server caught it. Three pods staying alive → a controller is watching.

Commands

```
kubectl get nodes / pods / svc / deployments / rs / all
kubectl logs <pod>
kubectl delete all --all
kubectl delete service <name>
kubectl apply -f <file>      # create OR update
kubectl create -f <file>     # create ONLY
```

🎯 INTERVIEW ALERT

apply vs create — she made them answer it:

	Object doesn't exist	Object already exists
create	✅ creates	❌ error
apply	✅ creates	✅ updates (or says <code>unchanged</code>)

"Apply can create plus update. Create will only create, not update."

🎯 Why ReplicaSet, Service and Deployment each exist

🗣️ SAID IN CLASS

"Everything I'm mentioning here — it's an interview question."

Why ReplicaSet? - "You can't create 100 pods with one `pod.yaml`. You can't do `kubectl apply pod1.yaml`, `pod2.yaml` ... You need multiple replicas." - **Load balancing** across those replicas - **Availability** — if one dies, another exists

Why Service? — and this is the answer that drives the whole class:

🔥 HARD MODE

The fact everything hinges on:

"If a pod goes into `CrashLoopBackOff`, or completes, it will create another pod — but it is **NOT** going to have the same IP address. Every time, a pod's internal IP address will change."

🔪 "You want to try? Take an `nginx` pod, make it crash, start a new pod — and you will see a new IP address."

```
pod dies      → 10.244.1.7  💀
pod restarts → 10.244.1.9  ← DIFFERENT IP
pod restarts → 10.244.2.3  ← DIFFERENT AGAIN
```

So your frontend **CANNOT** hardcode a backend IP. Ever.

✅ "That is one of the biggest risks we have, and that's why we need Service. It's not like every time we can use the IP address of the pod."

Why Deployment? — "Manage the `ReplicaSet`" · migrations · rollback · strategies.

🔊 INTERVIEW ALERT

And the killer: "What is the problem with *ReplicaSet*? We can't save the history. Have you ever seen history in a *ReplicaSet*? No — *ReplicaSet* will create, delete, update. That's it. If you want to save the history of previous deployments, that's why we need *Deployment*."

🔍 CLARIFIED

One small slip worth correcting. She said "the *ReplicaSet* automatically resolves the IP address from the name." It's actually the **Service** plus **CoreDNS** that do name → IP resolution. The *ReplicaSet* only keeps the right number of pods alive. Her point is right — a name resolves to the current pods — it's just a different component doing it. (And that's exactly what this class is about.)



02 🗂 Labels & Selectors — Where Each One Goes

She caught mistakes in the room and stopped to fix them properly.



Where	You write
Pod (and pod templates)	<code>labels</code>
Deployment	<code>selector</code> and <code>labels</code>
ReplicaSet	<code>selector</code> and <code>labels</code>
Service	<code>selector</code>

🗣 SAID IN CLASS

"Where are we putting labels? Inside `pod.yml` — so after deploying thousands of applications I can search my pod by a particular label. And where do we put the selector? *ReplicaSet*... not just *ReplicaSet* — inside the *Deployment* too. And in *Service* also. These three files are where we put selectors."

⚠️ The mistake she saw people making

⚠️ WATCH OUT

The label in your Service and the label in your Deployment **MUST** be identical.

```
# deployment.yaml      # service.yaml
spec:                  spec:
  template:            selector:
    metadata:           app: web-clusterip ← ✅ SAME
    labels:
      app: web-clusterip ← ✅ SAME
```

🗣️ "Why are both the labels and selectors the same in deployment and in service? Because we need to connect. How are you connecting your service with your deployment? To connect your service with the deployment, you are doing this."

✅ **Get one character wrong and your Service will have zero endpoints** — it'll exist, respond to nothing, and give you no error. Check with `kubectl get endpoints <service>`; empty means your labels don't match.

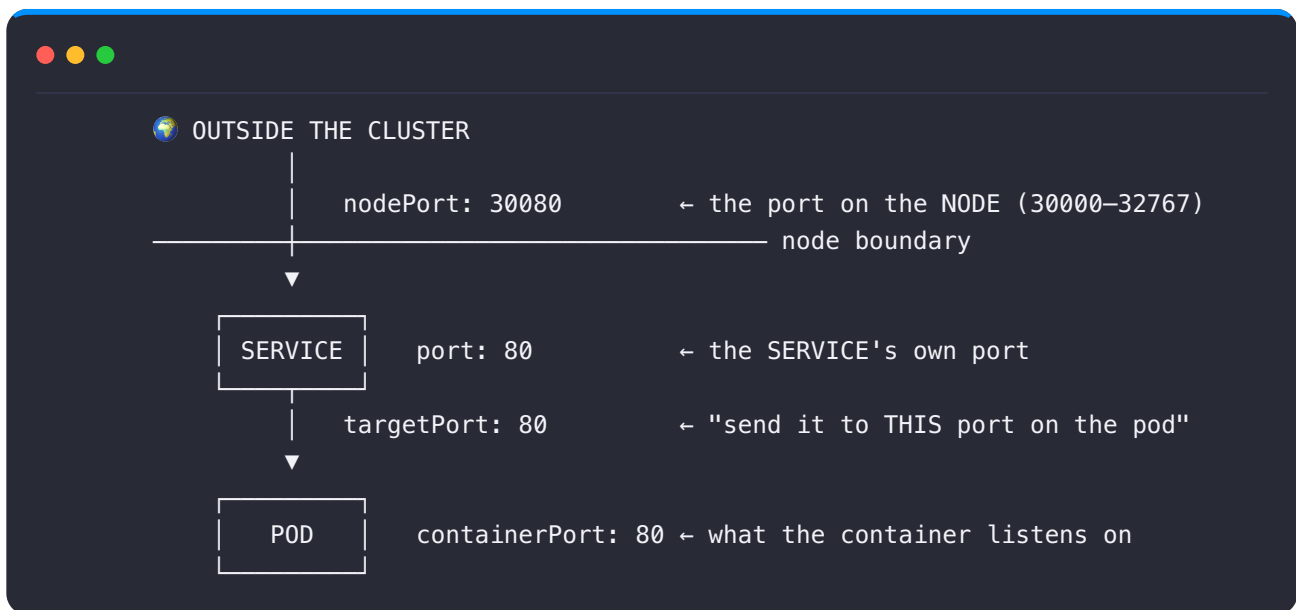
💡 GOOD TO KNOW

And it works with any workload: "It should be a *ReplicaSet*, a *Deployment*, a `pod.yml`, a *StatefulSet*, a *DaemonSet*. Anything can be there — but you need to attach your service to the core component."



03 ¹²/₃₄ The Four Ports — Untangled

She asked about these repeatedly because everyone mixes them up.



Field	Written in	Means
<code>containerPort</code>	Deployment / Pod	the port your app listens on inside the container
<code>targetPort</code>	Service	"forward to this port on the pod" — matches <code>containerPort</code>
<code>port</code>	Service	the Service's own port, used inside the cluster
<code>nodePort</code>	Service (NodePort type)	the port opened on every node , 30000–32767

✓ LOCK THIS IN

The two you always need: "The basic thing you need inside `ports` is `port` and `targetPort`. Two things by default."

💡 The resources aside — MiB vs MB, and millicores

She stopped on this because a student asked.

Written	Means
<code>1 GB</code>	1000 MB (<i>decimal</i>)
<code>1 GiB</code>	1024 MiB (<i>binary — what computers actually use</i>)
<code>128Mi</code>	128 mebibytes
<code>50m</code>	50 millicores = 0.05 of one CPU core

🗣 SAID IN CLASS

"CPU always we have in cores. So 50m means millicore — 50 millicore of CPU."

✓ `1000m` = 1 full core. So `250m` = a quarter of a core.

**04****1**

ClusterIP — The Default (Inside Only)

✓ LOCK THIS IN

"Cluster IP is the default service. It's an internal service — whatever we deploy, we can't see it in our browser directly unless we do port forwarding."

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
  labels:
    app: web-clusterip
spec:
  type: ClusterIP           # ← the default; you can omit it
  selector:
    app: web-clusterip      # ← must match the pod labels
  ports:
    - name: https
      protocol: TCP
      port: 8080             # the Service's port
      targetPort: 80         # the pod's port
```

💡 GOOD TO KNOW

"`name: https` and `protocol: TCP` are not necessary — I just put them by default so it'll be easy, so you learn some new tags. You can add more tags. Not even TCP — UDP also you can add."

🏆 Lab 1 — prove ClusterIP is internal-only

👤 DO THIS YOURSELF

This is the best demo of the class. Do all six steps.

Step 1 — deploy the app and the service:

```
minikube start
kubectl apply -f app-deployment.yaml
kubectl apply -f service.yaml
kubectl get all          # → 3 pods, 1 service, 1 replicaset, 1 deployment
```

Step 2 — deploy a second, throwaway pod just to test from:

```
kubectl apply -f client-pod.yaml
```

Her client pod is a **curl image** (`curlimages/curl:8.5.0`) running `sleep 3600` — "nothing it will do, it will just sleep for 3600 seconds, so your curl container will be up".

🧩 **Why `sleep` ?** A container dies when its main process ends (Class 6's PID 1 rule). `sleep 3600` gives you a container that stays alive for an hour doing nothing — a perfect debugging shell.

Step 3 — curl the service FROM INSIDE the cluster:

```
kubectl get svc          # copy the CLUSTER-IP
kubectl exec -it <curl-pod> -- curl -s http://<cluster-ip>:8080
```

✅ You get the nginx welcome page. It works.

Step 4 — now curl the SAME IP from your own terminal:

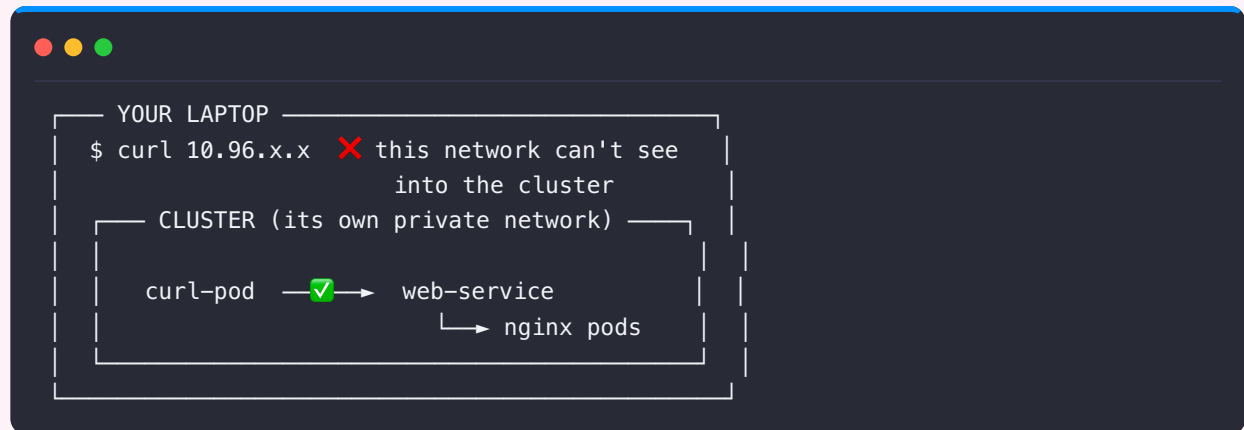
```
curl http://<cluster-ip>:8080
# → curl: (6) Could not resolve host / connection refused ❌
```

🎯 The question she built the whole demo around

🎯 INTERVIEW ALERT

"Why is the curl command not working? I'm able to curl from another container that I created, but I'm not able to from outside. Any idea?"

✅ **Her answer:** "The curl command we use in our normal terminal — it's inside my laptop's network. From a container we have a particular different internal network. That's why two containers in the same cluster are able to connect."



🧠 **Her sanity check:** "If I do `curl scaler.com` I get a response, because that website is up and reachable. But if a website is not visible — like localhost, this is my service, it's not visible — then curl will not work. Whatever you write in your terminal goes to your external network."

Step 5 — the escape hatch: port-forward

```
kubectl port-forward svc/web-service 8080:8080
# now open http://localhost:8080 ✅
```

🔑 "How to do port forwarding — `kubectl port-forward` and the service name, then `8080:8080`. And then you're able to see the nginx web page."

Step 6 — clean up:

```
kubectl delete all --all
```

✅ LOCK THIS IN

When to use ClusterIP: "For local use cases we are using ClusterIP." Internal service-to-service traffic — your backend talking to your database. Nothing outside should ever reach it.



05



2

NodePort — Open A Door On Every Node

✓ LOCK THIS IN

"Because it's a NodePort, I need to mention my node port — I'm not using the internal service network. I need to expose my service outside of my cluster."

```
spec:
  type: NodePort
  selector:
    app: web-server
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080      # ← the only new line
```

📌 NOTE

She says "380" throughout the recording — that's the transcriber losing digits. The real value is **30080**, and it must be in the **30000–32767** range. Kubernetes will reject anything outside it.

🎬 What changed from ClusterIP

🎬 WATCH IT HAPPEN

Two lines. That's the entire difference.

```
DEPLOYMENT FILE:  unchanged, except the name tag
                  (nginx-web → web-server)

SERVICE FILE:    type: ClusterIP → type: NodePort    ← change 1
                  + nodePort: 30080                  ← change 2
```

🗣️ *"Same image, same container port, same resources — everything is same. We just updated the pod name and the service type."*

🏆 Lab 2 — reach it from your browser, no port-forward

👤 DO THIS YOURSELF

Three ways to get the URL. Try them in this order.

```

kubectll apply -f app-deployment.yaml
kubectll apply -f service.yaml
kubectll get all

# Way 1 - minikube's IP + your nodePort
minikube ip                # e.g. 192.168.49.2
# → open http://192.168.49.2:30080

# Way 2 - let minikube build the URL for you ★ most reliable
minikube service web-service --url
# → http://127.0.0.1:60171      ← open this

# Way 3 - the cluster IP + port, from kubectll
kubectll get svc -o wide
```

😬 The Mac problem, and the fix

This ate a chunk of the class. Half the room saw the page; the Mac users didn't.

😊 CLASS MOMENT

"It's not working, let me check... Mac, it's not working, but Windows it should."

Then she found it: *"For Mac users, this is the port — 60171 — which Docker Desktop uses."*

🧩 WHY IT WORKS

Why Macs behave differently, since "it's a Mac thing" isn't an explanation.

On Linux, minikube's node IP is reachable from your machine directly. On **macOS and Windows**, Docker Desktop runs everything inside a **hidden Linux VM**, so `minikube ip` returns an address *inside that VM* which your browser can't route to.

✅ So `minikube service <name> --url` is the correct command on a Mac, not a workaround. It asks minikube to open a tunnel on a random localhost port (hence 60171) and hands you a URL that actually works.

```

LINUX:  browser  → minikube ip : 30080  ✓
MAC:    browser  → localhost:60171 → [VM] → :30080  ✓ (via --url)
        browser  → X → minikube ip      ✗
```

⚠️ WATCH OUT

And the port changes. "Docker will redirect every time to a new port." Re-run `--url` after every restart; don't bookmark it.

✅ LOCK THIS IN

When to use NodePort: "Whenever I want to do testing in my development environment, at that time I'm going to use it." Never in production — you're exposing a raw high-numbered port on every node.



06



3

LoadBalancer — Production's Answer

🎤 SAID IN CLASS

"What do you mean by load balancer?" Class: "Balancing the load. Redirect the traffic among all the pods, among all the microservices." ✅

```
spec:
  type: LoadBalancer
  selector:
    app: web-server
  ports:
    - port: 80
      targetPort: 80
```

🧠 What it actually gives you that NodePort doesn't

🧠 PICTURE THIS

Her framing, which is exactly right:

"In any cloud platform, whenever we create a load balancer, you get a URL. And from that URL you mention the port number and you see your application."

```
NODEPORT      → http://203.0.113.45:30080    ← a raw IP + weird port
LOADBALANCER  → http://my-app-1234.elb.amazonaws.com    ← a real URL
```

✅ "The only difference is you are using a URL instead of that public IP address. That is the only difference."

⚠️ On minikube it needs a tunnel

⚠️ WATCH OUT

"We cannot do it directly on our localhost unless we do the tunnel command, because it's going to create our load balancer locally. In actual environments we have a load balancer with a big URL. Because it's local, I just created the load balancer locally — only the name, not any URL."

👤 DO THIS YOURSELF

Lab 3 — LoadBalancer on minikube.

```
kubectl apply -f app-deployment.yaml
kubectl apply -f service.yaml
kubectl get svc          # → EXTERNAL-IP shows <pending> forever

# In a SECOND terminal, leave this running:
minikube tunnel          # ⚠️ asks for your laptop password

# Back in the first terminal:
kubectl get svc          # → EXTERNAL-IP now has an address ✅
minikube service web-service --url
```

🗣️ "There's a command to start a tunnel — `minikube tunnel`. It will ask for a password. You just give your PC password and then you're able to see your website. What it's doing is redirecting your minikube service IP and port to a new IP on your localhost."

⚠️ **Leave that terminal open.** Close it and the tunnel dies.

🧩 WHY IT WORKS

Why `EXTERNAL-IP` sits at `<pending>` without it: a LoadBalancer Service asks the **cloud provider** for a real load balancer. Your laptop has no cloud provider, so nobody answers. `minikube tunnel` pretends to be one.

Her troubleshooting ladder

WATCH OUT

"If you're not getting the URL on any laptop:"

- ```
1. kubectl delete all --all ← start clean
2. minikube tunnel ← then retry
3. still failing? CLOSE the terminal, open a new one, tunnel again
```

*"Once the tunnel command works, all the service issues you're getting will be resolved."*

### LOCK THIS IN

**When to use LoadBalancer:** *"Actual production environment. Because in production we can't expose my IP address — at that time I need some DNS URL."*



## 07 4 ExternalName — A Signpost, Not A Service

This one confuses people because it doesn't route traffic at all.

```
apiVersion: v1
kind: Service
metadata:
 name: external-api
spec:
 type: ExternalName
 externalName: api.github.com # ← just a DNS name
```

## 🎯 The one visible difference

### 🎬 WATCH IT HAPPEN

Run `kubectl get all` after each type and watch the IP column:

```
ClusterIP → CLUSTER-IP: 10.96.0.14 EXTERNAL-IP: <none>
NodePort → CLUSTER-IP: 10.96.0.21 EXTERNAL-IP: <none>
LoadBalancer → CLUSTER-IP: 10.96.0.33 EXTERNAL-IP: 127.0.0.1
ExternalName → CLUSTER-IP: <none> EXTERNAL-IP: api.github.com ★
```

🔗 "In external DNS you are not able to see the cluster IP, but you are able to see the external name. Can you see? `api.github.com`. This is the only change I wanted from you."

### 🧠 PICTURE THIS

**What it's actually for.** It's a **DNS alias inside your cluster**. Your app calls `external-api`, and CoreDNS quietly answers "that's `api.github.com`".

```
your pod → "external-api" —CoreDNS→ api.github.com
```

✅ **Why that's useful:** your code says `http://payments-api` in every environment. In dev that name points at a mock; in production it points at the real vendor. **You change the Service, never the code.**

⚠️ **It creates no proxy, no load balancing, and no pods.** It is purely a name → name redirect.

### ✅ LOCK THIS IN

**When to use it:** "When I have my own domain — not my client application, my own product, and I want my DNS for that product. At that time I'm using external name."

## 😬 The DNS experiment that failed

She tried to prove it end-to-end with a custom domain, editing `/etc/hosts` and testing with `nslookup`:

```
sudo nano /etc/hosts # add: 127.0.0.1 mydns.local
kubectl apply -f service.yml
kubectl exec -it <pod> -- nslookup <service> <service-ip>
→ server can't find ... ❌
```

## 😊 CLASS MOMENT

"It's not working. That means until and unless your DNS is synced up with your `/etc/hosts`, you are not able to see it."

Her conclusion: "You need to assume, guys — you need to assume that your DNS is already there. Once I get the IP address of my own website, we can perform this process. This domain is my domain, so I have my IP address and I can redirect it. Next session — we need a server."

💡 **She also offered it as an exercise:** "Anyone who has their own portfolio or deployed website — you can also try this."

## 🧩 WHY IT WORKS

**Why it couldn't work locally, briefly:** `/etc/hosts` on your laptop is read by *your laptop*. Pods inside the cluster don't read it — they ask **CoreDNS**. So editing `/etc/hosts` was never going to affect a lookup made from inside a pod. To do this properly you'd add a CoreDNS custom entry, or use a real registered domain — which is exactly why she deferred it to a session with a server.



08



5

## Headless — And Why StatefulSets Need It

### First: StatefulSet vs Deployment

## 🎯 INTERVIEW ALERT

"Whatever we are deploying through a Deployment — all are stateless."

### DEPLOYMENT (stateless)

web-app-7f8b9-x2k4l  
web-app-7f8b9-p9m2q  
web-app-7f8b9-k3n7w  
↑ random names

delete one ↓

web-app-7f8b9-z8t1r  
↑ a BRAND NEW random name

### STATEFULSET (stateful)

mysql-0  
mysql-1  
mysql-2  
↑ ORDERED, predictable

delete mysql-0 ↓

mysql-0  
↑ THE SAME NAME comes back

### SAID IN CLASS

"If I'm using a `StatefulSet`, it will create a particular name. Let's suppose MySQL — it will create `mysql-0`, `mysql-1`, `mysql-2`. Now if `mysql-0` is deleted, it will not create a pod with a random name. It will create `mysql-0` again. That is the only difference."

She proved it live:

```
kubectl delete pod mysql-0
kubectl get all
→ mysql-0 is back, "created just two seconds back" ✓ same name
```

### ✓ LOCK THIS IN

When to use which:

| Use for            |                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------|
| <b>Deployment</b>  | "frontend and backend components" — anything stateless                                    |
| <b>StatefulSet</b> | "MySQL, Postgres, MongoDB — at that time we cannot take a risk with stateless deployment" |

 "In organisations also we are using `StatefulSet` for our Postgres and MongoDB databases."

### NOTE

She also mentioned a third, out of syllabus: **DeploymentConfig** — "one of the advanced techniques". (That's an OpenShift-specific object, not vanilla Kubernetes.)

## | Now: the headless Service

```
spec:
 clusterIP: None # ★ THIS. That's the whole declaration.
 selector:
 app: mysql
```

## INTERVIEW ALERT

Her identification table — this is the interview question:

| If you see...                   | It's a...            |
|---------------------------------|----------------------|
| <code>type: ClusterIP</code>    | ClusterIP service    |
| <code>type: NodePort</code>     | NodePort service     |
| <code>type: LoadBalancer</code> | LoadBalancer service |
| <code>type: ExternalName</code> | ExternalName service |
| <code>clusterIP: None</code>    | ★ Headless service   |

 "Whenever you see cluster IP `none` , that means that service is a headless service. What do you mean by headless? It's one of the interview questions."

## WATCH OUT

"It is only specifically used for StatefulSets. You can't tell me you'll use `clusterIP: None` in your Deployment. No. Never."

## 🔍 One thing to correct before your interview

### 🎯 INTERVIEW ALERT

A student asked: "If clusterIP is none, am I able to access this service — even inside the cluster?"

Her answer: "No. You are not able to access it, because ClusterIP is an internal IP — if you block the internal IP then we're not able to access it directly."

🔍 **This one is worth getting right, because it's the opposite of how headless works — and it's precisely what gets asked.**

`clusterIP: None` does **not** block access. It means "don't give this Service one shared virtual IP." Instead, CoreDNS returns **the IP of every individual pod**, each with its own stable DNS name:

| NORMAL Service                                        | HEADLESS Service                                            |
|-------------------------------------------------------|-------------------------------------------------------------|
| mysql → 10.96.0.20<br>(one virtual IP,<br>random pod) | mysql → 10.244.1.5, 10.244.1.6, 10.244.1.7<br>(ALL pod IPs) |
|                                                       | mysql-0.mysql → 10.244.1.5 ★ address a SPECIFIC pod         |
|                                                       | mysql-1.mysql → 10.244.1.6                                  |
|                                                       | mysql-2.mysql → 10.244.1.7                                  |

✅ **And that's exactly why StatefulSets require it.** A database cluster needs to reach *the primary specifically*, not "whichever replica the load balancer picked". Stable names are the point — a virtual IP would destroy them.

🧠 "Headless" = no head (no single IP in front), not "no access".

📌 **For her assessment**, give her identification rule — `clusterIP: None` = headless, used with StatefulSet. That part is completely correct and is what's being tested.



## 09 🚒 FQDN — The Full Address Of A Service

She closed with this, and it's homework task 5 from last class.

|                |   |            |   |         |   |                |
|----------------|---|------------|---|---------|---|----------------|
| paymentservice | . | production | . | svc     | . | cluster.local  |
| └──┬──┘        |   | └──┬──┘    |   | └──┬──┘ |   | └──┬──┘        |
| service name   |   | namespace  |   | type    |   | cluster suffix |

## 🎯 INTERVIEW ALERT

### The problem it solves, in her words:

"When you have 10,000 services — how are you going to resolve the same service name in different environments? I have a dev environment, a prod environment, a test environment. I have a service named `backend` in dev. I have a service named `backend` in production. I have the same `backend` in test. Then how do I access the dev one?"

```
backend.dev.svc.cluster.local ← the dev one
backend.test.svc.cluster.local ← the test one
backend.production.svc.cluster.local ← the production one
```

✅ The namespace is what disambiguates them.

## 💡 GOOD TO KNOW

The short forms all work too — Kubernetes fills in the rest:

```
backend ← same namespace
backend.production ← another namespace
backend.production.svc.cluster.local ← fully qualified
```

🗣️ **And on the cluster suffix:** "If you're doing it with minikube you'll see `local`. If it's an AWS cluster, that cluster has its own DNS — I remember last time I deployed something, it was like `aws.eks...` something."



## 10 🎯 Which Service Type — The Decision Table

### 🗣️ SAID IN CLASS

"Even if you know all the types of services, but you don't know when you need to use which type — then it's not okay for you."

| Type      | Reachable from                            | Gives you            | ★ Use when                                           |
|-----------|-------------------------------------------|----------------------|------------------------------------------------------|
| ClusterIP | inside the cluster only                   | a stable internal IP | <b>default</b> — service-to-service, local use cases |
| NodePort  | outside, via<br><code>nodeIP:30080</code> | a port on every node | <b>testing in dev</b>                                |

| Type                                                | Reachable from             | Gives you                                | ★ Use when                                          |
|-----------------------------------------------------|----------------------------|------------------------------------------|-----------------------------------------------------|
| <b>LoadBalancer</b>                                 | outside, via a <b>URL</b>  | a cloud load balancer                    | <b>production</b> — "we can't expose my IP address" |
| <b>ExternalName</b>                                 | —                          | a <b>DNS alias</b> to something external | <b>you have your own domain / product</b>           |
| <b>Headless</b><br>( <code>clusterIP: None</code> ) | inside, <b>per-pod DNS</b> | one DNS name per pod                     | <b>StatefulSets</b> — databases                     |

#### 💡 GOOD TO KNOW

##### Her note on why you'd never find all five by Googling:

"Whenever you search 'how many types of Kubernetes services' on the internet, you'll find only three — ClusterIP, NodePort, LoadBalancer. If you do deep research you'll find the fourth, ExternalName. And after one or two years of using Kubernetes you'll find `clusterIP: None` — that's what we call a headless service."

🗣️ "Most DevOps engineers don't know there are five types. They only know four, and some people say only three."



## 11 📄 Command Cheat-Sheet

### Services

```
kubectl get svc # list services
kubectl get svc -o wide # + selector and extra detail
kubectl get endpoints <service> # ★ is anything actually behind it?
kubectl delete service <name>
kubectl port-forward svc/<name> 8080:80 # reach a ClusterIP from your laptop
```

### minikube specifics

```
minikube ip # the node's IP (Linux)
minikube service <name> # open it in your browser
minikube service <name> --url # ★ print the URL (works on Mac)
minikube tunnel # required for LoadBalancer – keep it
running
```

## SAID IN CLASS

"In minikube we do `minikube service` and the service name. And you need to type `service --svc` will not work. But in kubectl you can do `kubectl get svc`."

## Debugging from inside the cluster

```
kubectl exec -it <pod> -- curl -s http://<service>:<port>
kubectl exec -it <pod> -- nslookup <service>
kubectl exec -it <pod> -- sh
```

## GOOD TO KNOW

The **curl-pod trick** is worth keeping forever. Deploy `curlimages/curl` with `sleep 3600`, and you have a permanent shell inside the cluster network for testing whether services actually resolve.

## Service YAML shapes

```
ClusterIP (default)
type: ClusterIP
ports:
 - port: 8080
 targetPort: 80

NodePort
type: NodePort
ports:
 - port: 80
 targetPort: 80
 nodePort: 30080

LoadBalancer
type: LoadBalancer
ports:
 - port: 80
 targetPort: 80

ExternalName
type: ExternalName
externalName: api.github.com

Headless
clusterIP: None
```



## 12 Homework

### # Task

- 1  **Compare StatefulSet vs DaemonSet vs Deployment** — the one she stated explicitly
- 2  **Compare all five service types** — when to use each (§10)

## # Task

- 3 🏆 **Deploy all five service types yourself** and get the nginx page from each
- 4 🌐 Finish the **ExternalName / DNS** experiment if you have your own domain
- 5 📖 Read her [service.md](#) — *"I've clearly mentioned the difference... why we need service, how to use it, where we use it, what the problems are, and that's why we need different types"*
- 6 ⏳ Carried over: **Canary** and **Recreate** deployment strategies

## 🗣️ SAID IN CLASS

On her **service.md**: *"If you read this document, there is nothing left that you need to learn from any external website."*



## 13 📖 Notes on the Transcript Itself

- **Instructor name:** the header credits *"Ritesh Prajapati"* — the **Scaler++ Chrome extension's developer**, not the teacher. The instructor is **Nensi Ravaliya** ([@Nency-Ravaliya](#)).
- ⚠️ **The lecture title is wrong.** Folder and header both say *"Storage, HPA & Probes"* — **none of those were taught**. The class was entirely **Kubernetes Services** (all five types), plus StatefulSet vs Deployment and FQDN. Storage, HPA and Probes are still outstanding, as are **Ingress, ConfigMaps and Secrets** from Class 12.
- ⚠️ **The final ~25 minutes are largely unusable** — the failed DNS experiment and the closing lab dissolve into *"Thank you"*, Portuguese, Spanish, Indonesian and Hindi fragments. The DNS attempt (§7) is reconstructed from the readable parts; where the thread was lost it's marked rather than invented.
- 🔍 **Two clarifications added** rather than silent rewrites: **headless services are reachable inside the cluster** (§8 — the most important one, and the opposite of what was said), and **ReplicaSet doesn't do DNS resolution** — Service + CoreDNS do (§1). Her identification rule for headless is correct and is what the assessment will test.
- 📌 **nodePort** is heard as **"380"** throughout; the real value is **30080** (the legal range is 30000–32767).
- 📌 **The Mac networking problem** got a fuller explanation here (§5) — Docker Desktop's Linux VM — because *"it's a Mac issue"* doesn't tell you which command to reach for.
- Garbled terms corrected: **"Qublet"** → **kubelet**, **"Qproxy"** → **kube-proxy**, **"Qtxt / Qtl / cube ctl / kiftctl"** → **kubect1**, **"call / cull / crawl"** → **curl**, **"call images"** → **curlimages/curl**, **"noteport / notepad"** → **NodePort**, **"MiniCube / minicube"** → **minikube**, **"FQDM"** → **FQDN**, **"demon set"** → **DaemonSet**, **"exit command"** → **exec**, **"sbc / svc"** → **svc**, **"api.getup.com"** → **api.github.com**, **"external DNS"** → **ExternalName**, **"deployment config"** → **DeploymentConfig (OpenShift)**.